CSE 6220 High Performance Computing
Programming Assignment-3 Report
Submitted April 12, 2025

Name: Honglin Liu **gID:** 903651409
Name: Xuzheng Tian **gID:** 903618682

1 Implementation of the 2D SUMMA SpGeMM Algorithm

Our implementation of the Sparse General Matrix-Matrix Multiplication (SpGeMM) algorithm computes $C = A \times B$ for sparse matrices A ($m \times p$) and B ($p \times n$) in COO (Coordinate List Format) format, using a 2D $\sqrt{p} \times \sqrt{p}$ processor grid.
We adopt the Sparse SUMMA (Scalable Universal Matrix Multiplication) algorithm, optimized for non-zero elements, with key functions implemented in `SpGeMM.cpp`.

1.1 Algorithm implementation

第一步：要从root处理器分发数据

Spgemm:1. 计算block大小，并生成本地矩阵C，C的元素默认为0（不显式设置所有元素，当计算涉及某元素且其不存在时设为0）

The algorithm proceeds as follows:

1. **Initialization**:
   The `SpGeMM_2d` function determines the local block dimensions for each processor.
   For a processor at grid position (`row_rank`, `col_rank`), we compute block sizes as `block_size_m = (m+size-1)/size`, `block_size_n`, and `block_size_p`, where `size` is the grid dimension ($\sqrt{p}$).
   Actual block sizes (`current_size_m`, `current_size_n`, `current_size_p`) adjust for boundary conditions.

2. **COO to CSR Conversion**:
   To optimize non-zero element access, `COO2CSR` converts input matrices A and B from COO to CSR (Compressed Sparse Row Format) format.
   For a matrix with `lenth` non-zeros, `COO2CSR` allocates `row_ptr` (size `m+1`), `idx`, and `values` (size `lenth`).
   It counts non-zeros per row and computes a prefix sum for `row_ptr`, enabling efficient row-wise traversal.

3. **SUMMA Computation**:
   The `SUMMA` function executes the core multiplication.
   For each iteration k (0 to `col_size`):

   - **Broadcast** A:
     The k-th column block of A (CSR format) is broadcast within the row communicator

(row_comm) using MPI_Bcast.
The block includes row_ptr, idx, and values, with size determined by non-zero count (nnz_A).

- **Broadcast** B:
  The k-th row block of B is broadcast within the column communicator (col_comm).
- **Local Computation**:
  Each processor computes a contribution to Cs local block (C_block, a sparse map of coordinates to values).
  For each non-zero $A[i, A\_col]$ and $B[A\_col, B\_col]$, we update C_block[{i, B_col}] = plus(C_block[{i, B_col}], times(A[i, A_col], B[A_col, B_col])), where plus and times define the semiring.

C_block is initialized as an empty std::map in SpGeMM_2d, storing only non-zero entries.

4. **Output Conversion**:
   The MTX2COO function converts C_block to COO format.
   It iterates over non-zero entries in the sparse map, restoring global coordinates by adding offsets (row_rank * block_size_m, col_rank * block_size_n).
   The result is stored in C.

APSP：依然使用原来的spgemm_2d，这个忘记删除了，保留原来的逻辑，将0和没有声明的值都视为0.

1.2 Variations for APSP

For APSP, a variant SpGeMM_2d_apsp adjusts C_block handling.
Unlike SpGeMM_2d, it initializes C_block entries to 1000000 (for mimicking infinity) for the min-plus semiring and resets unchanged entries to 0 before conversion.
This ensures correct shortest path computation, avoiding issues with zero as the minimum in SpGeMM_2d.

The implementation ensures correctness for arbitrary $m \times p$ and $p \times n$ matrices while supporting flexible semiring operations, as tested with $A \times A^T$.

2 Implementation of the APSP Algorithm

The APSP (All-Pairs Shortest Path) algorithm computes shortest path distances between all vertex pairs in a weighted graph with n nodes, represented in COO format.
Inspired by the *Floyd-Warshall* algorithm, our implementation in `apsp.cpp` uses matrix multiplication with a *min-plus* semiring operator, executed via `SpGeMM_2d`.
The graph described in the problem is modeled as an $n \times n$ distance matrix L, where $L[i, j]$ is the edge weight from node i to j (0 denotes infinity).

2.1 Implementation of min-plus semiring

将所有的0元素视为正无穷：
意外情况：计算涉及对角线上的元素
1.对角线参与加和运算（aa+ab=ab):会使新的ab变为0（正无穷），原本ab或者消失或者被其他值取代
2.对角线参与最小值运算（min(aa,ak+ka)=ak+ka），原本的aa被取代

The *min-plus* semiring redefines operations:

- `plus`:
  $\min(a, b)$, selecting the shorter path.
  If $a = 0$ and $b = 0$, returns 0; if $a = 0$, returns b; if $b = 0$, returns a.
- `times`:
  $a + b$, summing edge weights.
  If $a = 0$ or $b = 0$, returns 0.

因此，需要在之后与前一步的结果做对比，填补缺失，替换最小值（3.post processing）

This allows $L^k[i, j]$ to represent the shortest path from i to j using at most k intermediate nodes, akin to Floyd-Warshalls iterative updates.

2.2 Algorithm implementation

The algorithm proceeds as follows:

1. **Initialization**:
   In `apsp`, we set L to the input graph and compute local block ranges for each processor (`row_rank`, `col_rank`) in the $\sqrt{p} \times \sqrt{p}$ grid.
   The block size is `block_size_n = (n+row_size-1)/row_size`, with row range [`row_start`, `row_end`) and column range [`col_start`, `col_end`).

2. **Iterative Matrix Multiplication**:
   For iterations where `max_iter` doubles from 1 to n:
   - Copy L to `L_tmp`.
   - Call `SpGeMM_2d` to compute $L =$ `L_tmp` $\times$ `L_tmp` under the min-plus semiring, updating $L[i, j] = \min(L[i, j], \min_k(L[i, k] + L[k, j]))$.

3. **Local Processing**:
   spgemm_2d 会返回全局位置，需要变回local位置方便计算
   Post-multiplication, each processor filters L to retain entries within its block, mapping global coordinates (i, j) to local (`i - row_start, j - col_start`).

A `std::map` merges `L_tmp` and L, keeping the minimum value for each coordinate to ensure correctness.

4. **Synchronization**:
   `MPI_Barrier` ensures processors align after each iteration.

5. **Output**:
   Global coordinates are restored by adding `row_start` and `col_start` offsets, yielding the result in COO format.

This *Floyd-Warshall*-inspired approach leverages `SpGeMM_2d`s parallel efficiency to compute shortest paths. The *min-plus* semiring ensures correctness, while local processing reduces communication overhead.

3 Visualization of Algorithms' Performances

We analyze the theoretical computational complexity and communication costs; with PACE clusters, we also record the actual performances of our SpGeMM (`SpGeMM_2d`) and APSP (`apsp`) implementations on a $\sqrt{p} \times \sqrt{p}$ processor grid with $P = 1^2, 2^2, 3^2, 4^2, 5^2, 6^2$.

3.1 Complexity and Communication

SpGeMM:

Comput. cost:
$O((\text{nnz}(A) \cdot \text{nnz}(B))/p + \sqrt{p})$ for one matrix multiplication.

Comm. cost:
$\sqrt{p}$ `MPI_Bcast` calls cost $O(\tau \cdot \sqrt{p} + \mu \cdot (\text{nnz}(A) + \text{nnz}(B))/\sqrt{p})$ per processor

APSP:

Comput.cost:
$O((\text{nnz}(G)^2 \cdot \log n)/p + \sqrt{p} \cdot \log n)$ for $\text{nnz}(G)$ edges, due to $\lceil \log n \rceil$ `SpGeMM_2d` calls.

Comm. cost:
$\log n \times$ SpGeMMs cost + `MPI_Barrier`, totaling $O(\tau \cdot \sqrt{p} \cdot \log n + \mu \cdot \text{nnz}(G) \cdot \log n/\sqrt{p})$ per processor.

*: nnz *abbr.* for "number of non-zero elements"; $G :=$ Graph; τ is latency; μ is bandwidth.

Comparison:
SpGeMM is lighter (single pass) but scales similarly to APSP per iteration. APSPs $\log n$ iterations inflate computation and communication, reducing scalability for large n.

3.2 Performance visualization

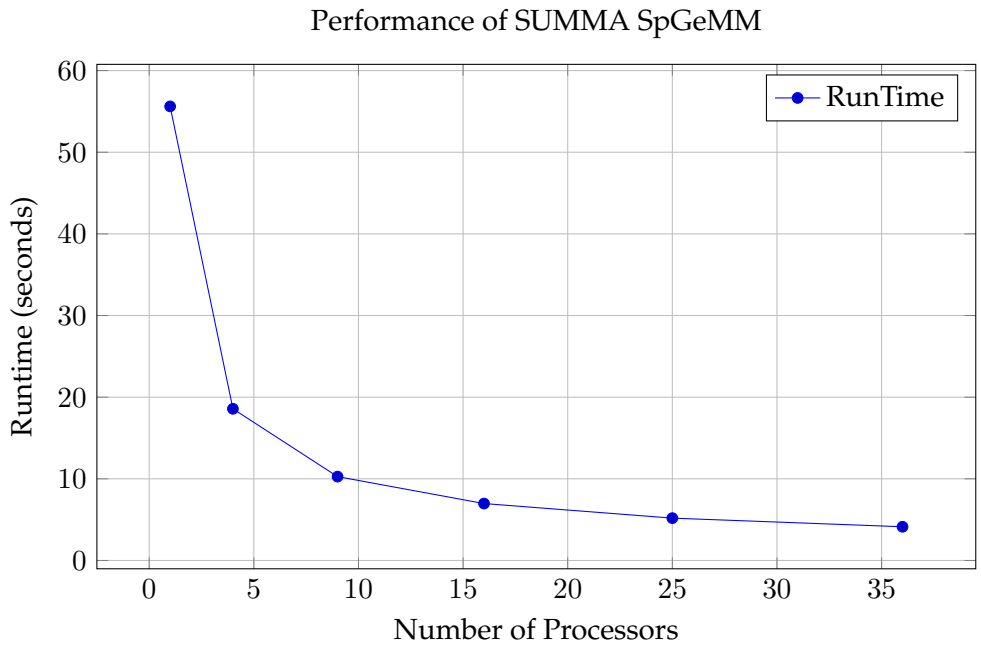

Figure 1: SpGeMM: 55.61s to 4.13s ($p = 36$, speedup 13.47×).

SpGeMM Scales from 55.61s ($p = 1$) to 4.13s ($p = 36$), meeting < 10s target. Speedup (13.47×) slows beyond $p = 16$ due to communication ($O(\alpha \cdot \sqrt{p})$).

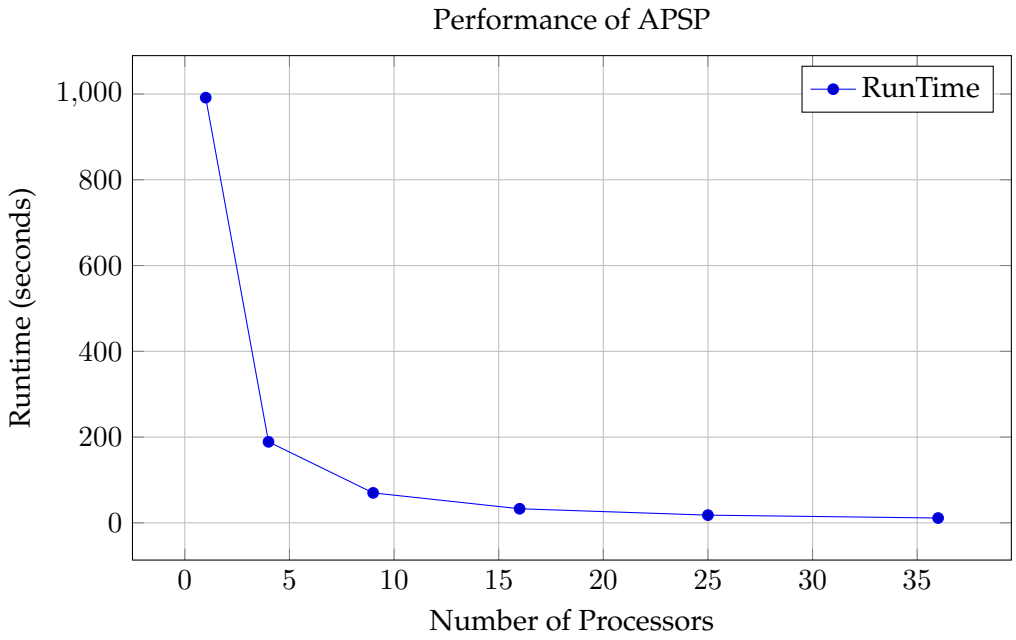

Figure 2: APSP: 991.48s to 11.28s ($p = 36$, speedup 87.94×).

APSP Drops from 991.48s to 11.28s, exceeding < 30s target. Higher speedup (87.94$\times$) reflects more enormous sequential workload, but $\log n$ iterations limit scalability.

APSPs iterative cost yields higher runtimes but better speedup. Both face load imbalance and communication bottlenecks at high p.